

Fecha de Entrega: 17 de marzo de 2026

Objetivos de Aprendizaje

Al completar este laboratorio, el estudiante será capaz de:

- Explicar las propiedades fundamentales de las funciones hash: unidireccionalidad, determinismo, efecto avalancha y resistencia a colisiones.
- Seleccionar el algoritmo hash correcto según el caso de uso: SHA-256 para integridad, Argon2id para contraseñas, HMAC-SHA256 para autenticación de mensajes.
- Implementar verificación de integridad de archivos usando SHA-256 con la biblioteca estándar de Python.
- Demostrar experimentalmente por qué MD5 y SHA-1 no deben usarse en sistemas modernos.
- Implementar almacenamiento seguro de contraseñas con Argon2id y salt, siguiendo guías OWASP.
- Construir un sistema de autenticación de mensajes (HMAC-SHA256) para APIs y demostrar la vulnerabilidad del patrón $H(\text{secret} || \text{mensaje})$.
- Diseñar y ejecutar una suite pytest que valide todos los componentes implementados.

Entregables:

- Repositorio Git con el código fuente completo
- README.md actualizado que incluya:
 - Descripción del proyecto
 - Instrucciones de instalación y uso
 - Ejemplos de ejecución
 - Respuestas a las preguntas de análisis
- Implementación: Código en el lenguaje de su preferencia (puede usar Jupyter Notebook si elige Python)

Recursos Recomendados

- Documentación del módulo random o secrets en Python
- Presentaciones de clase <https://locano-uvg.github.io/cifrados-26/>
- pycryptodome: <https://pycryptodome.readthedocs.io>
- RFC 8017 — PKCS#1 v2.2 (OAEP): <https://www.rfc-editor.org/rfc/rfc8017>

Escenario:

MediSoft S.A. distribuye software de diagnóstico a hospitales en Guatemala, Honduras y El Salvador. El software controla equipos de laboratorio críticos y debe cumplir con regulaciones de integridad de dispositivos médicos.

Recientemente, un hospital reportó que un paquete de actualización descargado desde un mirror no oficial modificó archivos de configuración. El análisis forense determinó que el atacante había comprometido el mirror e inyectado código malicioso.

Se les ha pedido que debe implementar dos capas de protección basadas en hashes:

- Integridad de distribución: verificar que los paquetes descargados son exactamente los que MediSoft publicó.
- Autenticación de usuarios: el portal de descarga almacena contraseñas de administradores de TI hospitalarios.

Problemas:

1. Comparación de algoritmos

Implementen el archivo `explorar_hashes.py`:

- Calculen el hash del string "MediSoft-v2.1.0" usando **MD5, SHA-1, SHA-256 y SHA-3/256**.
- Impriman una tabla comparativa mostrando:
 - Algoritmo
 - longitud en bits
 - longitud hex
 - el valor del hash.
- Calculen además el hash de "medisoft-v2.1.0" (solo cambió la capitalización) y muestren ambas filas juntas.
- Después de implementar, respondan:
 - ¿Cuántos bits cambiaron entre los dos hashes SHA-256? Usen XOR para contarlos. ¿Qué propiedad demuestra esto?
 - Con base en la longitud en bits, explica por qué MD5 es considerado inseguro para integridad de archivos

2. Generación de Claves

- Generen una lista de contraseñas comunes como:
 - ["admin", "123456", "hospital", "medisoft2024"]Para cada una, calcule SHA-256 y consulte si el hash aparece en el dataset público de **Have I Been Pwned (HIBP) usando su API**:
- https://api.pwnedpasswords.com/range/{primeros_5_chars}

Ejemplo:

```
admin = > 8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918
```

```
GET => https://api.pwnedpasswords.com/range/{8c697
```

- Impriman cuántas veces aparece cada hash en filtraciones conocidas

La API de HIBP usa el modelo k-Anonymity:

Nunca envíes el hash completo. Solo envías los primeros 5 caracteres del hash SHA-1 y recibes todos los sufijos que coinciden. El hash completo nunca sale de tu máquina.

Esto ilustra que SHA-256 directo sobre contraseñas es inseguro: estas contraseñas ya tienen sus hashes indexados en tablas rainbow públicas.

3. Verificación de integridad

MediSoft distribuye actualizaciones a hospitales a través de servidores espejo.

El riesgo: un atacante podría modificar los archivos en tránsito.

La solución es publicar junto al paquete un **manifiesto de hashes SHA-256** que permita al hospital verificar que cada archivo recibido es exactamente el que MediSoft publicó.

Implementen dos scripts que representan los dos lados de este proceso.

- Script **generar_manifiesto.py**
 - Simulen el rol de MediSoft al publicar un nuevo release.
 - El script debe recibir la ruta de al menos 5 archivos
 - Por cada calcular el SHA-256 correspondiente.
 - Creen un historial de hashes **SHA256SUMS.txt**
 - **Agreguen una nueva línea al archivo con cada <HASH> <nombre_archivo>**
- Script **verificar_paquete.py**
 - Simulen al administrador TI del hospital.
 - Su objetivo es validar cada uno de los archivos
 - Recalcula el SHA-256 de cada archivo listado y muestra un reporte de los archivos que fueron correctos y no correctos.
 - Como parte del ejercicio, modifiquen un byte de cualquier archivo del paquete, ejecuta el verificador y capturen la salida.

4. Implementación de Autenticidad - Generar firma

En el ejercicio anterior, MediSoft implementó un historial de hashes **SHA256SUMS.txt**. Sin embargo, un atacante sofisticado que logre comprometer el servidor podría no solo inyectar un paquete malicioso, sino también recalcular los hashes de esos archivos corruptos y reemplazar el archivo **SHA256SUMS.txt** con uno nuevo que coincida.

Para mitigar esto, MediSoft debe firmar digitalmente el historial utilizando criptografía asimétrica. De esta manera, el hospital puede comprobar criptográficamente que el manifiesto fue creado exclusivamente por MediSoft.

- Generación de Claves (generar_claves_rsa.py):
 - Utiliza la biblioteca pycryptodome para generar un par de claves asimétricas RSA de 2048 bits.
 - Exporten y guarden la clave **privada** en un archivo **medisoft_priv.pem**
 - Recordemos que las claves privadas NO SE DEBEN COMPARTIR
 - Exporten y guarden la clave **pública** en un archivo **medisoft_pub.pem**
 - Recordemos que las claves publicas SE PUEDEN COMPARTIR
- Firmar digital:
 - Calcular el hash SHA-256 del contenido del historial SHA256SUMS.txt.
 - Utiliza el esquema de firma estándar (por ejemplo, PKCS#1 v1.5 o PSS) para firmar el hash con la clave privada.
 - Guarda la firma resultante en un archivo binario llamado **SHA256SUMS.sig**

5. Verificación de Autenticidad - Validar firma

- Generen un script que realice lo siguiente:
 - Utilice la clave pública de MediSoft **medisoft_pub.pem**
 - Lea el historial de hashes **SHA256SUMS.txt**
 - Lea la firma generada **SHA256SUMS.sig**
- Implementen la validación de firma digital.
 - Implemente la validación de la firma
 - Muestren los resultados de la validación
- Abran el archivo **SHA256SUMS.txt** con un editor de texto y cambia un solo carácter del hash de cualquier archivo
- Muestren los resultados de la validación
- Alteración del Archivo de Datos
 - Abran uno de los archivos que utilizaron en el ejercicio 3 y modifiquen un byte.
 - Implementen la validación de firma digital
 - Respondan:
 - Porque la firma es valida? Que sucede al ejecutar **verificar_paquete.py**